

**TUGAS PRAKTIKUM 3**  
**SISTEM OPERASI**



**Dosen Pengampu Mata Kuliah:**  
**Triawan Adi Cahyanto, M.K**

**Disusun Oleh:**

**Nama : Muhammad Fahrur Rohman**  
**Nim :2410651056**  
**Prodi : Teknik Informatika**  
**Tanggal : 30 Oktober 2025**

**PROGRAM STUDI TEKNIK INFORMATIKA**  
**FAKULTAS TEKNIK**  
**UNIVERSITAS MUHAMMADIYAH JEMBER**

**2025**

## Tugas Praktikum 3

### Tugas 1: Kalkulator Paralel

Jadi, disini kita akan membuat sebuah program kalkulator yang awalnya menerima sepasang angka dari user sebagai input yang kemudian akan diproses dan ditampilkan output dari penjumlahan, pengurangan, perkalian, dan pembagian dari kedua angka tersebut yang dikerjakan oleh thread berbeda.

Pertama-tama, buat sebuah file program bernama "nano kalkulator\_paralel" dengan isi kode seperti pada gambar, kemudian compile dan jalankan.

```
#include <stdio.h>
#include <pthread.h>

// Variabel global untuk menyimpan hasil
float a, b;
float hasil_tambah, hasil_kurang, hasil_kali, hasil_bagi;

// Fungsi untuk setiap operasi aritmatika
void* operasi_tambah(void* arg) {
    hasil_tambah = a + b;
    pthread_exit(NULL);
}

void* operasi_kurang(void* arg) {
    hasil_kurang = a - b;
    pthread_exit(NULL);
}

void* operasi_kali(void* arg) {
    hasil_kali = a * b;
    pthread_exit(NULL);
}

void* operasi_bagi(void* arg) {
    if (b != 0)
        hasil_bagi = a / b;
    else
        printf("Kesalahan: Pembagian dengan nol tidak diperbolehkan!\n");
    pthread_exit(NULL);
}

int main() {
    pthread_t t1, t2, t3, t4;

    printf("=== Kalkulator Paralel ===\n");
    printf("Masukkan angka pertama: ");
    scanf("%f", &a);
    printf("Masukkan angka kedua: ");
    scanf("%f", &b);

    // Membuat empat thread paralel
    pthread_create(&t1, NULL, operasi_tambah, NULL);
    pthread_create(&t2, NULL, operasi_kurang, NULL);
    pthread_create(&t3, NULL, operasi_kali, NULL);
    pthread_create(&t4, NULL, operasi_bagi, NULL);

    // Menunggu seluruh thread selesai
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    printf("\n=== Hasil Perhitungan ===\n");
    printf("Penjumlahan : %.2f\n", hasil_tambah);
    printf("Pengurangan : %.2f\n", hasil_kurang);
    printf("Perkalian : %.2f\n", hasil_kali);
    if (b != 0)
        printf("Pembagian : %.2f\n", hasil_bagi);

    return 0;
}
```

```

fahru@FAHRUR:~/praktikum3/tugas$ nano kalkulator_paralel.c
fahru@FAHRUR:~/praktikum3/tugas$ gcc kalkulator_paralel.c -o kalkulator_paralel -pthread
fahru@FAHRUR:~/praktikum3/tugas$ ./kalkulator_paralel
=== Kalkulator Paralel ===
Masukkan angka pertama: 15
Masukkan angka kedua: 5

=== Hasil Perhitungan ===
Penjumlahan : 20.00
Pengurangan : 10.00
Perkalian   : 75.00
Pembagian   : 3.00

```

Jadi, gambar di atas merupakan hasil perhitungan dari empat thread yang masing-masing menjalankan operasi hitung berbeda, semuanya dilakukan secara parallel, program utama menunggu semua thread selesai melakukan operasi hitung sebelum hasil ditampilkan, dan menggunakan `pthread_exit()` dan `pthread_join()`. Program barusan termasuk kedalam multithreading. Untuk penjelasan detail kodenya ada dibawah ini.

Penjelasan kode:

`#include <stdio.h>` (ini digunakan untuk fungsi input/output)

`#include <pthread.h>`(dan ini itu library khusus linux untuk mengelola thread)

`float a, b;` (kode disamping yaitu a dan b adalah variabel angka yang akan diinput user untuk kemudian dihitung)

`float hasil_tambah, hasil_kurang, hasil_kali, hasil_bagi;` (sedangkan yang disamping ini digunakan untuk menyimpan hasil dari tiap operasi hitung yang dilakukan)

// yang dibawah ini adalah untuk menghitung hasil pertambahan antara a dan b, terus disimpan ke hasil tambah

```

void* operasi_tambah(void* arg) {
    hasil_tambah = a + b;
    pthread_exit(NULL);
}

```

//yang ini sama saja seperti diatas, hanya saja yang ini pengurangan

```

void* operasi_kurang(void* arg) {
    hasil_kurang = a - b;
}

```

```
    pthread_exit(NULL);  
}
```

//Sama juga, tapi ini perkalian

```
void* operasi_kali(void* arg) {  
    hasil_kali = a * b;  
    pthread_exit(NULL);  
}
```

//kalau yang ini pembagian, hanya saja ada tambahan untuk mengecek apakah b bukan 0 karena jika 0, operasi pembagian tidak akan dilakukan untuk mencegah error divided by zero.

```
void* operasi_bagi(void* arg) {  
    if (b != 0)  
        hasil_bagi = a / b;  
    else  
        printf("Kesalahan: Pembagian dengan nol tidak diperbolehkan!\n");  
    pthread_exit(NULL);  
}
```

//Kalau ini dipakai untuk mendeklarasikan empat variabel pthread\_t untuk mewakili 4 thread berbeda yang tiap threadnya melakukan operasi hitung yang berbeda.

```
int main() {  
    pthread_t t1, t2, t3, t4;
```

//Ini hanya keterangan untuk input angka, anggap saja interfacenya.

```
    printf("=== Kalkulator Paralel ===\n");  
    printf("Masukkan angka pertama: ");  
    scanf("%f", &a);  
    printf("Masukkan angka kedua: ");  
    scanf("%f", &b);
```

```

// Membuat empat thread paralel

pthread_create(&t1, NULL, operasi_tambah, NULL);
pthread_create(&t2, NULL, operasi_kurang, NULL);
pthread_create(&t3, NULL, operasi_kali, NULL);
pthread_create(&t4, NULL, operasi_bagi, NULL);


// Ini program untuk menunggu seluruh thread selesai untuk mencegah program menampilkan
hasil thread yang belum selesai bekerja

pthread_join(t1, NULL);
pthread_join(t2, NULL);
pthread_join(t3, NULL);
pthread_join(t4, NULL);

//Dibawah ini anggap saja seperti wadah untuk menampilkan dan memberi keterangan hasil proses
hitung.

printf("\n=== Hasil Perhitungan ===\n");
printf("Penjumlahan : %.2f\n", hasil_tambah);
printf("Pengurangan : %.2f\n", hasil_kurang);
printf("Perkalian : %.2f\n", hasil_kali);
if (b != 0)
    printf("Pembagian : %.2f\n", hasil_bagi);

//Ini hanya untuk mengakhiri program

return 0;

}

```

### Percobaan 1.1: Membuat Proses Sederhana

Cara : Buat file process\_basic.c dengan isi kode seperti berikut, lalu compile dan jalankan hingga hasilnya Nampak seperti pada gambar terakhir.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    printf("Program dimulai\n");
    printf("PID proses ini: %d\n", getpid());

    1 / 14

    printf("PID parent: %d\n", getppid());

    return 0;
}
```

```
pid_t pid;
printf("Program dimulai\n");
printf("PID proses ini: %d\n", getpid());
-bash: syntax error near unexpected token '('
pid_t: command not found
-bash: syntax error near unexpected token '"Program dimulai\n"'
-bash: syntax error near unexpected token '"PID proses ini: %d\n",'
fahru@FAHRUR:~$ nano process_basic.c
fahru@FAHRUR:~$ gcc process_basic.c -o process_basic
./process_basic
Program dimulai
PID proses ini: 1435
PID parent: 274
```

Fungsi Percobaan 1.1 adalah untuk memahami bagaimana sebuah program yang dijalankan menjadi proses aktif dengan PID unik, menampilkan hubungan antara proses dan parent-nya melalui `getpid()` dan `getppid()`, serta menjadi dasar dalam mempelajari cara pembuatan dan pengelolaan proses di sistem operasi.

## Tugas 2: File Processing

Pada tugas kedua ini, kita akan membuat sebuah program yang membaca file teks dan menghitung jumlah katanya menggunakan 2 thread yang kemudian hasil antar threadnya digabung karena sebelumnya, threadnya dibagi menjadi 2 bagian, dan kemudian dibandingkan dengan yang hanya 1 thread. Pertama-tama, buat file program bernama “file\_processing.c” dengan isi kode seperti pada gambar, buat file txt dengan isi kata bebas, lalu compile dan jalankan programnya.

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>
#include <time.h>

#define MAX_SIZE 10000

typedef struct {
    char* buffer;
    int start;
    int end;
    int count;
} ThreadData;

// Fungsi untuk menghitung kata dalam bagian tertentu
int hitung_kata_serial(char* buffer, int start, int end) {
    int inWord = 0;
    int count = 0;

    for (int i = start; i < end; i++) {
        char c = buffer[i];
        if ((c == ' ' || c == '\n' || c == '\t') && !inWord) {
            inWord = 0;
        } else if (c != ' ' && c != '\n' && c != '\t' && !inWord) {
            inWord = 1;
            count++;
        }
    }

    return count;
}

// Fungsi versi multi-thread
void* hitung_kata(void* args) {
    ThreadData* data = (ThreadData*) args;
    data->count = hitung_kata_serial(data->buffer, data->start, data->end);
    pthread_exit(NULL);
}

int main() {
    FILE* file = fopen("teks.txt", "r");
    if (!file) {
        printf("Gagal membuka file.\n");
        return 1;
    }

    char* buffer = malloc(MAX_SIZE);
    int length = fread(buffer, 1, MAX_SIZE, file);
    fclose(file);

    printf("=== Perhitungan Kata (Perbandingan Single vs Multi-thread) ===\n\n");

    // ===== VERSI SINGLE THREAD =====
    clock_t start_single = clock();
    int total_single = hitung_kata_serial(buffer, 0, length);
    clock_t end_single = clock();
    double waktu_single = (double)(end_single - start_single) / CLOCKS_PER_SEC;

    printf("Single-thread: Jumlah kata = %d\n", total_single);
    printf("Waktu eksekusi: %.6f detik\n\n", waktu_single);

    // ===== VERSI MULTI THREAD =====
    int tengah = length / 2;
    ThreadData t1 = { buffer, 0, tengah, 0 };
    ThreadData t2 = { buffer, tengah, length, 0 };

    pthread_t thread1, thread2;

    clock_t start_multi = clock();
    pthread_create(&thread1, NULL, hitung_kata, &t1);
    pthread_create(&thread2, NULL, hitung_kata, &t2);
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    clock_t end_multi = clock();

    int total_multi = t1.count + t2.count;
    double waktu_multi = (double)(end_multi - start_multi) / CLOCKS_PER_SEC;

    printf("Multi-thread : Jumlah kata = %d\n", total_multi);
    printf("Waktu eksekusi: %.6f detik\n\n", waktu_multi);

    free(buffer);
    return 0;
}

```

```
fahru@FAHRUR: ~/praktikum x + v
GNU nano 6.2 disclaimer.txt
Assalamu'alaikum warahmatullahi wabarakatuh
Halo
Yth. Bapak/Ibu atau siapa saja yang membaca dan menilai tugas praktikum kali ini, saya, Muhammad Fahrur Rohman
dengan NIM 2410651056 dari prodi Teknik Informatika, menyatakan bahwa saya mengerjakan tugas praktikum ini
dengan tidak sepenuhnya menggunakan AI dan saya mmengerjakannya masih dibarengi dengan kemampuan saya sendiri dalam
menjelaskan dan merangkai kata. Maka dari itu, apabila jika di pendeteksi AI terdeteksi bahwa tugas saya ini sebagian
besar dikerjakan oleh AI, maka hal tersebut tidak bisa dibenarkan dan tidak akurat. Oleh karena itu, saya mohon kpd
bpk/ibu sekalian agar mempertimbangkan ulang keputusan beliau sekalian dalam memberi nilai
dan tidak memberi nilai minim terhadap saya.

[ Wrote 9 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^G Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo      M-6 Copy

fahru@FAHRUR:~/praktikum3/tugas$ nano file_processing.c
fahru@FAHRUR:~/praktikum3/tugas$ gcc file_processing.c -o file_processing -pthread
fahru@FAHRUR:~/praktikum3/tugas$ ./file_processing
=== Penghitungan Kata (Perbandingan Single vs Multi-thread) ===

Single-thread: Jumlah kata = 105
Waktu eksekusi: 0.000006 detik

Multi-thread : Jumlah kata = 105
Waktu eksekusi: 0.000411 detik
```

Bisa dilihat hasil eksekusi pada gambar diatas, jumlah kata pada keduanya benar dan valid, hanya saja ada perbedaan lama waktu eksekusi dikarenakan jumlah thread yang berbeda.

### Tugas 3: Producer-Consumer

Pada tugas kali ini, kita akan membuat sebuah program simulasi antara produser dan konsumen dengan sifat thread produser menghasilkan angka acak yang kemudian angkanya diproses oleh konsumen dengan menggunakan mutex untuk sinkronisasi dan buffernya maksimal 10 item. Caranya ialah buat file program bernama "producer\_consumer.c" dengan isi kode seperti pada gambar, kemudian compile dan jalankan.



```

#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define BUFFER_SIZE 10

int buffer[BUFFER_SIZE];
int count = 0; // jumlah item dalam buffer

pthread_mutex_t mutex;

void* producer(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);

        if (count < BUFFER_SIZE) {
            int item = rand() % 100;
            buffer[count] = item;
            count++;
            printf("Producer menghasilkan item: %d (jumlah item: %d)\n", item, count);
        } else {
            printf("Buffer penuh! Producer menunggu...\n");
        }

        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
    pthread_exit(NULL);
}

void* consumer(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);

        if (count > 0) {
            int item = buffer[count - 1];
            count--;
            printf("Consumer mengambil item: %d (jumlah item: %d)\n", item, count);
        } else {
            printf("Buffer kosong! Consumer menunggu...\n");
        }

        pthread_mutex_unlock(&mutex);
        sleep(2);
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t prod, cons;
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    pthread_mutex_destroy(&mutex);
    return 0;
}

```

Untuk detailnya kurang lebih seperti ini:

### 1. Import library

#include <stdio.h> → Dipakai untuk fungsi input/output seperti printf().

#include <stdlib.h> → Digunakan sebagai fungsi rand() yang menghasilkan angka acak.

#include <pthread.h> → Agar bisa menggunakan thread dan mutex.

#include <unistd.h> → Sebagai fungsi sleep() agar ada jeda waktu antar proses.

### 2. Definisi konstanta dan variabel global

#define BUFFER\_SIZE 10 (ukuran maksimum buffer (tempat penyimpanan sementara)).

#define TOTAL\_ITEM 10 (jumlah total item yang akan diproduksi dan dikonsumsi (agar program berhenti otomatis setelah selesai))

int buffer[BUFFER\_SIZE]; array berisi data yang disimpan sementara.

int count = 0; (jumlah item saat ini di dalam buffer (0 berarti kosong, 10 berarti penuh))

### 3. Deklarasi mutex

pthread\_mutex\_t mutex;

- mutex adalah kunci pengaman (mutual exclusion) agar hanya satu thread yang bisa mengakses buffer pada satu waktu.
- Tanpa mutex, race condition bisa terjadi (producer dan consumer bisa menulis/menghapus data bersamaan → hasil kacau).

### 4. Fungsi producer

pthread\_mutex\_lock(&mutex) pada fungsi producer diunakan untuk mengunci akses ke buffer agar thread producer yang hanya bisa mengubah data di saat itu.

```
if (count < BUFFER_SIZE) {
```

```
    int item = rand() % 100;
```

```
    buffer[count] = item;
```

```
    count++;
```

```
    printf("Producer menghasilkan item: %d (jumlah item: %d)\n", item, count);
```

```
}
```

- Mengecek apakah buffer belum penuh (`count < BUFFER_SIZE`).
- Jika belum penuh:
  - Membuat item acak antara 0–99.
  - Menyimpannya ke buffer di indeks terakhir.
  - Menambah count untuk menunjukkan jumlah item sekarang.
  - Menampilkan hasilnya ke layar.

```
pthread_mutex_unlock(&mutex);
```

```
sleep(1);
```

```
}
```

```
pthread_exit(NULL);
```

```
}
```

- `pthread_mutex_unlock(&mutex)` → membuka kunci agar thread lain (consumer) bisa mengakses buffer.
- `sleep(1)` → memberi jeda 1 detik biar output bergantian dan mudah dibaca.
- `pthread_exit(NULL)` → menandakan bahwa thread producer sudah selesai setelah memproduksi 10 item.

## 5. Fungsi consumer

```
void* consumer(void* arg) {
```

```
    for (int i = 0; i < TOTAL_ITEM; i++) {
```

```
        pthread_mutex_lock(&mutex);
```

- Sama seperti producer, consumer juga mengunci akses buffer agar tidak bentrok dengan producer.

```
        if (count > 0) {
```

```
            int item = buffer[count - 1];
```

```
            count--;
```

```
            printf("Consumer mengambil item: %d (jumlah item: %d)\n", item, count);
```

```
}
```

- Mengecek apakah buffer tidak kosong ( $\text{count} > 0$ ).
- Jika ada data:
  - Mengambil item terakhir di buffer ( $\text{buffer}[\text{count} - 1]$ ).
  - Mengurangi count (karena 1 item diambil).
  - Menampilkan hasil ke layar.

```
pthread_mutex_unlock(&mutex);
```

```
sleep(2);
```

```
}
```

```
pthread_exit(NULL);
```

```
}
```

- `pthread_mutex_unlock()` → membuka kunci buffer lagi setelah mengambil item.
- `sleep(2)` → jeda 2 detik, supaya consumer berjalan lebih lambat dari producer (biar interaksinya jelas).
- `pthread_exit()` → mengakhiri thread setelah selesai mengonsumsi 10 item.

## 6. Fungsi main

```
int main() {
```

```
pthread_t prod, cons;
```

```
pthread_mutex_init(&mutex, NULL);
```

- Membuat dua variabel thread (prod dan cons).
- Menginisialisasi mutex dengan `pthread_mutex_init()` supaya siap digunakan.

```
pthread_create(&prod, NULL, producer, NULL);
```

```
pthread_create(&cons, NULL, consumer, NULL);
```

- Membuat dua thread:
  - prod menjalankan fungsi `producer()`.
  - cons menjalankan fungsi `consumer()`.
- Keduanya berjalan secara bersamaan (concurrent).

```
pthread_join(prod, NULL);
```

```
pthread_join(cons, NULL);
```

- Menunggu kedua thread selesai sebelum program utama lanjut ke akhir.
- Tanpa pthread\_join(), program utama bisa berakhir duluan sebelum thread selesai.

```
pthread_mutex_destroy(&mutex);
```

```
printf("\nProses selesai!\n");
```

```
return 0;
```

```
}
```

- Setelah semua thread selesai, mutex dihancurkan (destroy) agar tidak ada memori yang tertinggal.
- Menampilkan teks “Proses selesai!” untuk menandakan bahwa simulasi berakhir.

```
disclaimer.txt file_processing file_processing.c kalkulator_paralel kalkulator_paralel.c producer_consumer.c
fahru@FAHRUR:~/praktikum3/tugas$ gcc producer_consumer.c -o producer_consumer -pthread
fahru@FAHRUR:~/praktikum3/tugas$ ./producer_consumer
Producer menghasilkan item: 83 (jumlah item: 1)
Consumer mengambil item: 83 (jumlah item: 0)
Producer menghasilkan item: 86 (jumlah item: 1)
Consumer mengambil item: 86 (jumlah item: 0)
Producer menghasilkan item: 77 (jumlah item: 1)
Producer menghasilkan item: 15 (jumlah item: 2)
Consumer mengambil item: 15 (jumlah item: 1)
Producer menghasilkan item: 93 (jumlah item: 2)
Producer menghasilkan item: 35 (jumlah item: 3)
Consumer mengambil item: 35 (jumlah item: 2)
Producer menghasilkan item: 86 (jumlah item: 3)
Producer menghasilkan item: 92 (jumlah item: 4)
Consumer mengambil item: 92 (jumlah item: 3)
Producer menghasilkan item: 49 (jumlah item: 4)
Producer menghasilkan item: 21 (jumlah item: 5)
Consumer mengambil item: 21 (jumlah item: 4)
Consumer mengambil item: 49 (jumlah item: 3)
Consumer mengambil item: 86 (jumlah item: 2)
Consumer mengambil item: 93 (jumlah item: 1)
Consumer mengambil item: 77 (jumlah item: 0)
Proses selesai!
```

Dari gambar hasil eksekusi program diatas, bisa kita lihat bahwa producer dan consumer bekerja secara bergantian, producer menambah data ke buffer dan kemudian diambil oleh consumer satu-satu. Tidak ada dua thread yang mengakses buffer secara bersamaan karena sudah dikunci dengan mutex.

## Kesimpulan

Secara garis besar, kita banyak belajar memahami konsep dasar dari thread, penerapan paralelisme thread dalam menjalankan tugas demi efisiensi, serta menerapkan sinkronisasi antar thread.